

Faculty of Computers and Data Science
Intelligent Systems Department

Speech Recognition project
Arabic Text-to-Speech (TTS) System

Text To Speech...



Team Members

Name	ID
Ahmed Mostafa AbdEl-Rahman	20221372883
AbdelRahman Mohamed Abdel-Hadi	2022513643
Mazen Gaber Mahmoud	20221372110
AbdelRahman Tarek Zaki	20221442265

Contents

Abstract	4
1. Introduction	4
2. Methodology	4
2.1 Data Collection	4
1. User Configuration	4
2. Searching for Videos	4
3. Filtering Videos with Arabic Transcripts	5
4. Downloading Audio and Subtitles	5
5. Converting Audio to WAV	5
6. Metadata Collection	5
7. Automation Workflow	5
Note on Transcription Issues	5
2.2 Data Preprocessing	6
1. Renaming Files for Consistency	6
2. Segmenting Audio Files	6
3. Audio Preprocessing	6
4. Transcribing Processed Audio	7
5. Splitting and Transcription Conversion to phonemes	7
2.3 Model Training	8
1. Cloning the Repository	8
2. Starting Model Training	8
3. Changes Made in the Configuration Files	9
3. Results and Evaluation	9
3.1 Metrics	Error! Bookmark not defined.
3.2 Observations	Error! Bookmark not defined.
4. Challenges	9

Abstract

This project created a Text-to-Speech (TTS) system for Arabic using audio from YouTube. We focused on Arabic and used deep learning with a model called Tacotron 2. The process included cleaning the data, transcribing audio, and converting it into phonemes for training. The final model produced clear and natural speech.

1. Introduction

TTS systems turn written text into speech. Arabic is challenging for TTS because of its different dialects and unique sounds. This project aimed to:

1. Build a TTS model for Arabic.
 2. Develop an easy-to-follow process for preparing the data.
 3. Train and test a model that works well for Arabic phonetics.
-

2. Methodology

2.1 Data Collection

The data collection phase focuses on extracting high-quality audio and corresponding Arabic subtitles from YouTube videos. This phase involves searching for videos, filtering those with Arabic transcripts, downloading their audio, and converting the audio into a standardized format. Below is a detailed explanation of the process:

1. User Configuration

The script starts with user-defined settings:

API Key: Used to access the YouTube Data API for searching videos.

Search Queries: A list of Arabic-focused keywords, such as *أخبار عربية* ("Arabic News") and *اللغة العربية الفصحى* ("Classical Arabic"), to ensure a diverse range of topics and dialects.

Target Video Count: The desired number of videos to process, set at 500.

Directories: Paths for storing downloaded videos, processed WAV files, and transcriptions are specified and created if they don't already exist.

2. Searching for Videos

- Searches YouTube using the specified queries and filters videos by duration (5–20 minutes) for better content variability.
- Returns a list of unique video IDs up to the target count (500).

3. Filtering Videos with Arabic Transcripts

The `has_arabic_transcript` function ensures that only videos with Arabic subtitles are processed:

- It checks for available transcripts in Arabic using the `YouTubeTranscriptApi` library.
- Videos without Arabic transcripts are excluded, reducing unnecessary downloads.

4. Downloading Audio and Subtitles

The `download_video_and_subs` function handles downloading:

- Downloads only the audio portion of a video using the `yt-dlp` tool.
- Saves automatically generated Arabic subtitles in the VTT format (if available).

5. Converting Audio to WAV

To standardize audio input for training, the `convert_to_wav` function:

- Converts downloaded audio files to WAV format.
- Ensures a sample rate of 16 kHz and single-channel audio, which is commonly required for TTS models.

6. Metadata Collection

After processing each video:

- The audio file (WAV format) and subtitles are moved to their respective directories.
- Metadata, including the video ID, YouTube URL, subtitle file path, and WAV file path, is recorded in a CSV file for further use.

7. Automation Workflow

The `main` function automates the entire workflow:

1. Searches for videos using the specified queries.
2. Filters videos with Arabic transcripts.
3. Downloads and processes audio and subtitle files.
4. Stores the metadata in a structured format.

Key Output

- Processed audio files in WAV format, ready for model training.
- Corresponding Arabic subtitles for alignment with audio.
- A metadata CSV file linking video IDs to their processed assets.

Note on Transcription Issues

During the data collection phase, an issue was observed with the transcription files retrieved from YouTube videos. When converting these subtitle files into plain text, the resulting text often contained repeated words and

phrases. This repetition is likely caused by the way YouTube's automatic transcription system segments subtitles for display purposes.

2.2 Data Preprocessing

The data preprocessing phase involves several crucial steps to prepare the collected audio and transcription data for use in training the Arabic Text-to-Speech (TTS) model. Below are the details of the process:

1. Renaming Files for Consistency

- **Objective:** To rename WAV files and corresponding transcription files for a uniform naming convention (e.g., `001.wav`, `002.wav`).
 - **Steps:**
 - Read all WAV files in the specified directory.
 - Generate a numerical base name for each file with leading zeros (e.g., `001`, `002`).
 - Rename WAV files and move them to a new directory.
 - Check for corresponding transcription files (`.vtt`) and rename them using the same base name. If a transcription file is missing, a warning is issued.
 - **Outcome:** A clean and organized dataset with uniformly named audio and transcription files.
-

2. Segmenting Audio Files

- **Objective:** To split audio files into smaller segments, each up to 15 seconds long, matching the duration requirements for the TTS model.
 - **Steps:**
 - Load each WAV file and determine its duration.
 - Parse the corresponding transcription file to extract subtitles and their timestamps.
 - Split the audio into segments of up to 15 seconds. Ensure timestamps align with the transcriptions.
 - Save each audio segment with a descriptive name (e.g., `001_0001.wav`) and write the associated transcription to a text file.
 - **Outcome:** Smaller audio segments with aligned text transcriptions, ready for model training.
-

3. Audio Preprocessing

- **Objective:** To standardize the audio files for optimal model compatibility.
- **Steps:**
 - Load each audio file and resample it to the target sampling rate (e.g., 22,050 Hz).
 - Normalize the audio signal to ensure consistent amplitude across all files.
 - Trim silence from the start and end of the audio to remove unnecessary pauses.
 - Append a fixed amount of silence at the end of each segment to ensure smooth transitions.
 - Save the processed audio files to a designated directory.
- **Outcome:** Cleaned and normalized audio files with consistent properties.

4. Transcribing Processed Audio

- **Objective:** To generate text transcriptions for the audio segments using the Whisper model.
 - **Steps:**
 - Load a pre-trained Whisper model (e.g., `medium` size) capable of recognizing Arabic.
 - Iterate through all processed WAV files.
 - Transcribe each file using the Whisper model, specifying the target language as Arabic (`"ar"`).
 - Save the generated transcriptions in a `.txt` format, named to match the corresponding audio file.
 - **Outcome:** High-quality text transcriptions of audio files, providing the textual data required for TTS model training.
-

5. Splitting and Transcription Conversion to phonemes

1. Collecting WAV and Transcription Pairs

- **Objective:** To pair audio files (`.wav`) with their corresponding transcription files (`.txt`).
 - **Steps:**
 - Iterate through the directory of processed WAV files.
 - For each WAV file, check if a matching transcription file exists.
 - Read the transcription text and pair it with the WAV filename.
 - If no transcription is found, the file is skipped with a warning.
 - **Outcome:** A list of (audio file, transcription text) pairs is created, ensuring only valid pairs are included.
-

2. Splitting the Data into Training and Testing Sets

- **Objective:** To divide the dataset into training and testing sets based on a specified ratio.
 - **Steps:**
 - Shuffle the list of collected pairs to ensure randomization.
 - Split the data based on the defined training ratio (e.g., 90% for training and 10% for testing).
 - **Outcome:** Two separate datasets—one for training and another for testing—ready for further processing.
-

3. Writing Arabic Transcriptions

- **Objective:** To save the Arabic transcriptions in a structured format.
- **Steps:**
 - Write the training and testing sets to separate text files (`train_arab.txt` and `test_arab.txt`).
 - Each line follows the format: `"audio_filename.wav" "Arabic transcription"`.
- **Outcome:** Arabic transcription files for both training and testing datasets.

4. Converting Arabic to Buckwalter Transliteration

- **Objective:** To transliterate Arabic text into the Buckwalter format.
 - **Steps:**
 - Use a custom function (`arabic_to_buckwalter`) to convert each Arabic transcription into Buckwalter.
 - Write the results to separate text files (`train_buckw.txt` and `test_buckw.txt`).
 - Each line follows the format: `"audio_filename.wav" "Buckwalter transcription"`.
 - **Outcome:** Buckwalter transliteration files for training and testing datasets.
-

5. Converting Buckwalter to Phoneme Sequences

- **Objective:** To generate phoneme sequences from the Buckwalter transliteration for use in training the TTS model.
 - **Steps:**
 - Read the Buckwalter transcription files (`train_buckw.txt` and `test_buckw.txt`).
 - For each Buckwalter transcription, use a conversion function (`buckwalter_to_phonemes`) to generate the corresponding phoneme sequence.
 - Save the phoneme sequences in separate text files (`train_phon.txt` and `test_phon.txt`).
 - Each line follows the format: `"audio_filename.wav" "phoneme_sequence"`.
 - **Outcome:** Phoneme sequence files for training and testing datasets.
-

2.3 Model Training

The following outlines the key steps and configuration adjustments for training the Arabic Text-to-Speech (TTS) model:

1. Cloning the Repository

- **Objective:** Obtain the necessary codebase for Arabic TTS model training.
 - **Steps:**
 - Clone the repository:
 - `!git clone https://github.com/nipponjo/tts-arabic-pytorch.git`
 - Copy the repository to Google Drive for persistent storage:
 - `!cp -r /content/tts-arabic-pytorch /content/drive/MyDrive/`
-

2. Starting Model Training

- **Objective:** Train the TTS model using the provided scripts and configuration files.

- **Steps:**
 - Navigate to the project directory in Google Drive:
 - `%cd /content/drive/MyDrive/tts-arabic-pytorch`
 - Start training with a specified configuration file:
 - `!python train.py --config './configs/nawar_tc2_adv.yaml'`
-

3. Changes Made in the Configuration Files

Key Adjustments in `nawar_tc2_adv.yaml`:

- **Restoring a Pretrained Model:**
 - Initially, the parameter `restore_model` was empty (`restore_model: ""`), meaning no pretrained model was loaded.
 - After completing 4 epochs, the parameter was updated to load a saved checkpoint:
 - `restore_model: /content/drive/MyDrive/tts-arabic-pytorch/checkpoints/exp_tc2_adv/epoch_4_states.pth`
 - This ensures training resumes from the last saved state.

- **Number of Iterations for Saving States:**

Increased `n_save_states_iter` from **10 iterations** to **200 iterations**, reducing the frequency of state-saving to focus more on the training process.

- **Dataset Paths:**

Updated dataset paths in the configuration file to point to the preprocessed data stored in the working environment.

Key Adjustments in `basic.yaml`:

1. **Epoch Settings:**
 - Reduced the total number of training epochs from **500** to **5** for the initial training phase.
 2. **Batch Size Adjustments:**
 - Initially set the batch size to **8** for the first five epochs.
 - After five epochs, increased the batch size to:
 - **32** to leverage higher GPU memory and accelerate training.
 - **64** in the fifth epoch for even faster processing.
 - Decreased the batch size back to **8** in the final epochs to stabilize training.
-

3. Results and Evaluation

Observations:

1. **Model Trained from Scratch (5 Epochs):**
 - The generated sound was of poor quality, with noticeable noise.
 - The spectrogram displayed low red color, which may indicate weak signal strength.

- The waveform plot was dominated by blue, suggesting low energy or silence.
- 2. **Model Trained with Pretrained Weights (5 Epochs):**
 - The sound produced was still of poor quality, starting with a low volume, followed by silence.
 - The spectrogram showed low red color in the first 100 frames, possibly indicating low signal activity during that period.
 - The waveform plot displayed a blue line that was initially medium height, but later thinned out, suggesting a lack of sustained sound energy.

Simple Explanation:

- **Low red color in spectrogram:** The red color often indicates stronger signal or energy. Low red suggests weak sound energy or poor quality.
- **Blue waveform plot:** Blue typically represents low-amplitude or silence. A blue line suggests little or no sound energy.

Conclusion:

Both models exhibited poor audio quality and weak signal representation in the spectrogram and waveform plots. The model trained from scratch showed complete lack of meaningful sound, while the pretrained model displayed a low volume and an absence of sustained sound. Further adjustments in training, data, and model architecture are needed to improve the sound quality and energy consistency.

Challenges:

1. **Transcription Quality Issues (Phase 1):**

When attempting to transcribe videos sourced from YouTube, the resulting transcriptions contained invalid characters, rendering them unsuitable for conversion to the Buckwalter format.
2. **Repeated Words in Transcriptions (Phase 1):**

Videos with existing YouTube captions provided transcriptions in VTT files. However, after converting these files to TXT format, the resulting text contained numerous repeated words, requiring significant preprocessing.
3. **Memory Limitations with Batch Size:**

Increasing the batch size to 32 caused memory exhaustion. To address this, we utilized a GPU with higher memory capacity to support larger batch sizes.
4. **Training Time:**

The model required 2 to 3 hours to train a single epoch, making the training process time-intensive and resource-demanding.